

MoE Serving: Thesis Proposal

Plan A: Profile-Guided Receiver-Aware Rank-LUT Partial Combine

Plan B: Joint Energy-Efficiency and SLO-Constrained Optimization for MoE Inference

June 16, 2026

Plan A

Profile-Guided Receiver-Aware Rank-LUT Partial Combine

Positioning

Existing MoE communication optimizations focus almost exclusively on dispatch and expert placement. What is overlooked in the combine phase is not the transmission itself, but a unique degree of freedom it offers: gate weights serve as importance signals that are freely available, amenable to differential treatment, and structurally infeasible to exploit during dispatch.

Core Argument

Structural Thesis: In MoE, the combine phase is the only stage where gate-aware differential-precision transmission is viable.

- Dispatch Phase: Each token is replicated k times and dispatched to top- k experts. All replicas share the same hidden state. Imposing different precisions on different replicas according to their gate values would require k independent quantization passes at distinct precisions - either wasteful or disruptive to the regularity of the collective.
- Combine Phase: Each (token, expert) pair produces an independent output $o_{t,e}$, with $g_{t,e}$ already known. Varying precision by gate weight is structurally regular. This difference is determined by the MoE computation graph structure, not by an engineering implementation trick.

The mathematical form of combine: $y_t = \sum_{e \in S(t)} g_{t,e} o_{t,e}$, which is linear and recombinaible.

- y_t : the hidden state of token t after combine; $S(t)$: the set of experts selected by routing ($|S(t)|=k$ under top- k)
- $g_{t,e}$: gate weight of expert e for token t (softmax-normalized, $\sum_{e \in S(t)} g_{t,e}=1$); $o_{t,e}$: the d -dimensional output of expert e

Claims

#: C1

Prerequisite Claim: The combine contribution $g_{t,e}$; Existing Literature & Assessment: $o_{t,e}$; Col 4: exhibits a pronounced long-tail distribution within the top-k set, rather than only a "mild imbalance"; Col 5: No direct support; only indirect evidence. MoDES (2025) and Not All Experts are Equal (ACL 2024) demonstrate expert-level imbalance (most experts in the global expert pool are unimportant) - a holistic-level conclusion, not one within top-k. A characterization study can measure the per-token contribution distribution within top-k. A priori, the larger top-k is (e.g., top-6 in DeepSeek-V2-Lite), the smaller the tail contribution, but empirical measurement is required to draw conclusions. Verification is split across two orthogonal layers: (i) C1 itself (the $g_{t,e}$; Col 6: $o_{t,e}$; Col 7: long-tail) must be validated by attaching forward hooks on real MoE models (e.g., DeepSeek-V2-Lite, Qwen-MoE) to directly sample per-(token, expert) contributions and plot their distribution - this is beyond what AICB can provide; (ii) Alibaba's open-source AICB (AI Communication Benchmark) (<https://github.com/aliyun/aicb>) is used to generate/analyze communication traces, estimate the load distribution across receiver ports, and supply frequency inputs $\text{freq}(l,r,R)$ to the optimization model. These two responsibilities are orthogonal.

#: C2

Prerequisite Claim: A substantial portion of end-to-end accuracy degradation comes from routing drift (perturbed hidden states alter downstream routing decisions, amplified across layers), rather than pure numerical deviation; Existing Literature & Assessment: EAQuant (2025, arxiv 2506.13329) indirectly corroborates the existence of this risk. That work identifies "Routing Fragility Under Quantization Noise" as one of the three major challenges in MoE quantization, stating: "router's top-k expert selection is highly sensitive to quantization-induced perturbations, causing misrouting and cascading degradation," and addressing it by "aligning routing and eliminating drift." However, EAQuant does not report the fraction of total accuracy loss attributable to drift. The positioning of experiments in this paper: first quantify "the total accuracy loss after applying the scheme" (baseline figure), then borrow EAQuant's "routing alignment" idea to decompose the loss into drift vs. numerical error components, answering what fraction drift contributes to combine-approximation loss - rather than pre-claiming drift as the dominant cause.

C2 Experimental Design

C2 in fact quantifies two distinct phenomena and must not be conflated:

1. Total accuracy degradation quantification (baseline) : First, perform the simplest comparison - two forward passes under no information loss vs. approximation/dropping as prescribed by this scheme - and directly measure the end-to-end accuracy gap, to inform the reader: what is the total accuracy loss after applying this idea? This figure is the one reviewers care most about and is mandatory.
2. Accuracy degradation attribution (routing drift decomposition) : On top of (1), perform attribution -
 - Experiment A: approximated forward pass + gate freely selects experts
 - Experiment B: approximated forward pass + gate locked to the original model's routing
 - The accuracy gap between the two serves as an approximate estimate of the routing drift contribution (locking routing itself alters the model's execution semantics, so it is not a strict equality; however, it captures the same order of magnitude of drift impact). The remainder is approximately attributed to pure numerical error.
 - This decomposition draws on EAQuant's "routing alignment" idea and answers the question: what exactly causes the accuracy loss?

Statically Deployable Strategy

Decompose the decision into three mutually orthogonal dimensions that can be determined offline:

- **WHERE** | Triggered by receiver-port congestion: Assume the network core has sufficient resources and is congestion-free; all contention occurs only at receiver ports. The more congested a given receiver port (the more senders sharing it, the more aggregated expert outputs), the more aggressively precision is reduced at that receiver - for example, if receiver *r* simultaneously receives outputs for experts 2, 5, and 7, the one with the highest rank number (i.e., $R=k$, the expert with the lowest gate weight) gets reduced precision when the port is congested, whereas all maintain full precision when idle. The optimization objective is directly anchored to "P99 utilization / queue length at each receiver port." Intra-node NVLink is minimally affected by congestion and defaults to full precision.

For deployability, to avoid binding the LUT to a specific cluster / placement, the receiver dimension is offline-clustered into receiver groups (three tiers: hot / warm / cold, partitioned by the quantiles of aggregated expert traffic). The actual LUT key is (layer_id, receiver_group, rank).

Re-deployment only requires re-running the receiver-group classification once to reuse the LUT.

- **WHICH-LAYER** | Filter by sensitivity: Run an end-to-end offline experiment, applying approximation to each layer in isolation and measuring perplexity, to obtain a layer-wise sensitivity heatmap. Low-sensitivity layers enable approximation; high-sensitivity layers retain full precision. When entering the optimization model, high-sensitivity layers are directly fixed to BF16, and the Rank-LUT is solved only over low-sensitivity layers.

Statically Deployable Strategy (cont.)

- HOW | Rank-LUT static table lookup (quantization first, dropping second) : At runtime, only a table lookup determines precision (BF16 / FP8 / INT4 / drop) - $O(1)$, no sorting, no online optimization, no per-token decisions. Uses rank (i.e., the position of the expert within the token's top-k, R in $1, \dots, k$) as the importance proxy - rank is already determined at routing time, requiring neither gate-quantile estimation nor online threshold maintenance, making it the simplest to deploy; rank 1 = highest gate, rank k = lowest gate (the paper uniformly refers to this as "the highest-numbered rank / lowest-ranked expert / $R=k$ " to avoid ambiguity around the term "lowest rank"), consistent with the long-tail direction of C1. The quantization path is fixed-width, regular, and unbiased, making it the easiest to deploy. Dropping is used only at the highest-numbered rank position ($R=k$), with gate renormalization $y_{\text{approx}}(1)/(\sum_{e \in \text{keptg_t}, e} \sum_{e \in \text{keptg_t}, e} \text{eo_t}, e)$ applied as a best-effort correction (note: this is an approximate compensation for the scale bias introduced by dropping, not mathematically unbiased; its effectiveness depends on the long-tail property of C1), at zero communication cost. The lookup key is (layer_id, receiver_group, rank), where receiver_group is the result of offline-clustering receiver ports by aggregated traffic (e.g., hot/warm/cold), decoupling the LUT from specific clusters / placements for cross-deployment reuse. The gate-bucket variant is evaluated separately as an ablation / enhanced variant in the evaluation section (rank is a coarsened proxy for gate values; when Rank-LUT already approaches the oracle upper bound, gate bucketing becomes non-essential).

The Unified Optimization Problem

Decision Variables (One-Hot Form)

For each triple (layer l , receiver group r , rank R), introduce a set of 0-1 decision variables:

$x_{l,r,R,p}$ in $0,1$, p in $P=BF16, FP8, INT4, drop$

sum_p in P $x_{l,r,R,p}=1$ for all (l,r,R)

For high-sensitivity layers, fix $x_{l,r,R,BF16}=1$; the MILP optimizes precision selection only over low-sensitivity layers.

Index semantics:

Index: l

Meaning: The l -th MoE layer; Range / Example: 1,2,dots, L

Index: r

Meaning: Receiver group (offline-clustered by aggregated per-port traffic); Range / Example: r in hot, warm, cold (three tiers); LUT decoupled from specific GPU ids

Index: R

Meaning: Rank (the position of the expert within the token's top- k); Range / Example: R in 1,2,dots, k , already determined at routing time

Index: p

Meaning: Precision level; Range / Example: $P=BF16, FP8, INT4, drop$

Decision Variables (One-Hot Form) (cont.)

$x_{l,r,R,p}=1$ means: during the combine phase of layer l , all (token, expert) pairs whose expert output is destined for receiver group r and whose rank is R shall be transmitted uniformly at precision p .

Each triple selects exactly one of the 4 precision levels. The bytes function:

$$\text{bytes}(l,r,R)=\sum_p \text{in } P \text{size}(p) * x_{l,r,R,p}$$

where $\text{size}(p)$ is a constant (BF16 = 2 B / FP8 = 1 B / INT4 = 0.5 B / drop = 0 B).

Optimization Problem

The objective directly targets receiver-port congestion - minimize the maximum utilization across all receiver groups:

$$\min_{x,U} U \text{ s.t. } \lambda_{r(x)}/\mu_r \leq U \text{ for all } r \text{ in hot, warm, cold}$$

where $\lambda_{r(x)} = \sum_{l,R,p} \text{psize}(p) * x_{l,r,R,p} * \text{freq}(l,r,R)/T_{\text{step}}$ is the byte arrival rate for receiver group r (normalized by the representative port within the group), μ_r is the bandwidth of ports in that group, and $\text{freq}(l,r,R)$ comes from an offline trace (the number of (token, expert) pairs at layer l destined for receiver group r with rank R). Introducing the auxiliary variable U keeps the formulation as MILP.

Constraints:

- Accuracy constraint (per-(layer, rank, precision) profile table) : During offline profiling, $\delta_{l,R,p}$ is defined as the marginal accuracy degradation contribution per (token, expert) pair - i.e., "when one (token, expert) pair at rank R in layer l switches to precision p , the marginal increment in end-to-end accuracy degradation relative to the all-BF16 baseline (worst-case upper bound; $\delta_{\text{equiv. 0}}$ for BF16)." Note that $\delta_{l,R,p}$ is independent of receiver group r - accuracy degradation is determined by (layer, rank, precision), with the receiver-group dimension entering the constraint only through frequencies. The constraint is written in linear form:

$$\sum_{l,r,R,p} \delta_{l,R,p} * x_{l,r,R,p} * (\text{freq}(l,r,R)) / (\sum_{l',r',R'} \text{freq}(l',r',R')) \leq \epsilon$$

i.e., "the global weighted-average marginal accuracy degradation induced by the decisions $\leq \epsilon$," with consistent dimensionality. In this way, the differences among INT4 vs. drop, rank=1 vs. rank=k, and sensitive vs. insensitive layers all fall into the profile table, allowing the decision variables to precisely target combinations such as "R=k + hot receiver group" for precision reduction. At calibration time, compare the offline-predicted loss "when all layers and all ranks use p : $\sum w * \delta$ " against the measured loss, to verify (i) dimensional consistency and (ii) the inter-layer additivity bias as a sanity check.

Optimization Problem (cont.)

- TBT constraint (receiver-port utilization upper bound) : Since the objective is already min-max utilization, this constraint participates directly as $U \leq \rho^*$ (ρ^* e.g., 0.7, corresponding to an empirical upper bound on P99 queuing delay). The end-to-end TBT is decomposed as $TBT_{p99} = T_{bar_attn} + T_{bar_compute}(MoE\ FFN) + T_{bar_dispatch} + T^{queue_combine}(x) + T_{bar_other}$, where the constant terms are obtained offline and the combine queuing is the sole tunable term; T_{other} includes layer normalization, residual paths, sampling, and runtime overhead. Intra-node NVLink is minimally affected by congestion and defaults to full precision.

Variable scale: $O(L_{low} |groups| \times k |P|)$, where L_{low} is the number of low-sensitivity layers filtered by layer sensitivity analysis; high-sensitivity layers are fixed to BF16 and excluded from the MILP. For example, 16 low-sensitivity layers $\times$ 3 receiver groups $\times$ top-2 routing $\times$ 4 precision choices yields about 384 binary variables; even top-6 remains around 1.1k binary variables. With only 3 receiver groups and rank up to k , the formulation remains small-scale MILP.

Oracle Upper Bound (offline, post-hoc; not integrated into runtime)

On a logged trace, allow each (token, expert) to freely select its precision:

$$b^*_{t,e} = \operatorname{argmin}_b \text{ bytes s.t. } \|y_{t,e}^{\text{approx}} - y_{t,e}^{\text{full}}\|_2^2 \leq \delta$$

The oracle uses combine-output MSE (i.e., the L2 distance between the approximated combine output $y_{t,e}^{\text{approx}}$ and the all-BF16 output $y_{t,e}^{\text{full}}$) as a local constraint - this is a quantity that is genuinely decomposable to individual (token, expert) precision selections (each perturbation to $o_{t,e}$ maps directly to a perturbation to $y_{t,e}$). Logit/KL divergence, being an end-to-end result after subsequent layers, is not naturally decomposable and is used only as a post-hoc correlation calibration: report the correlation coefficient between "combine MSE oracle gain" and "end-to-end PPL gain" to validate the effectiveness of the local proxy. Finally, report $\text{gap} = \text{static gain} / \text{oracle gain}$ - the closer to 1, the closer the static strategy approaches the theoretical upper bound. Runtime still requires only a table lookup keyed by (layer, receiver_group, rank), with $O(1)$ zero overhead.

Evaluation Plan

- Baselines: (i) all-BF16 combine (no compression); (ii) uniform FP8 combine; (iii) uniform INT4 combine; (iv) rank-only heuristic (no receiver awareness); (v) receiver-only heuristic (no rank differentiation); (vi) Rank-LUT (this work); (vii) offline oracle (upper bound).
- Metrics: end-to-end TBT P99 / mean; perplexity on WikiText-2 and a long-context benchmark; combine-stage byte savings; gap-to-oracle ratio.
- Models & Setup: DeepSeek-V2-Lite (top-6) and Qwen1.5-MoE-A2.7B (top-4); single-node 8xA100/H100 with expert parallelism; vLLM or SGLang as the serving backend.

Risks & Fallback

- C1 long-tail not pronounced enough (rank-k contribution > 10%): drop the drop precision tier; keep only BF16/FP8/INT4 differential quantization. The receiver-aware + Rank-LUT structure remains intact and the bytes-saving story is weaker but still valid.
- Routing drift dominates the loss: tighten $\delta_{l,R,p}$ to a per-layer cap and consider EAQuant-style routing alignment as an add-on.
- MILP solve time too high in real clusters: fall back to LP-relaxation + rounding, or solve per-layer independently.

Datasets, Models & Tools

- MoE models: DeepSeek-V2-Lite (top-6) and Qwen1.5-MoE-A2.7B (top-4) for forward-hook contribution sampling and end-to-end perplexity.
- Language-quality benchmarks: WikiText-2 (PPL) and a long-context benchmark such as LongBench or PG19 (long-form PPL).
- Communication traces: AICB (Alibaba AI Communication Benchmark) for receiver-port load estimation and freq(l,r,R) inputs.
- Serving backend: vLLM or SGLang on a single node with 8 x A100 / H100, expert parallelism enabled.
- Reference work: EAQuant (arxiv 2506.13329, 2025) for routing-fragility motivation; MoDES (2025) and Not All Experts are Equal (ACL 2024) for expert-level imbalance evidence.

Plan B

Joint Energy-Efficiency and SLO-Constrained Optimization for MoE Inference

Positioning

Existing MoE serving systems optimize almost exclusively for latency / throughput. Energy optimization for dense models has been explored extensively, but the knobs are largely "instance count / parallelism degree / GPU frequency" - an MoE-specific energy model is essentially absent, and MoE provides several additional expert-level knobs. This plan constructs an MoE-specific energy model (comprising static, dynamic computation, and communication components) and jointly optimizes expert placement and replication count under SLO constraints, with the objective of minimizing J/token.

Core Insight: MoE Offers Two Additional Expert-Level Knobs Beyond Dense Models

Dense models are limited to knobs such as "instance count / parallelism degree / GPU frequency"; MoE provides two extra knobs unavailable to dense models:

1. Expert placement - determines the communication energy cost of all-to-all;
2. Expert replication count - trades off "increased multi-GPU static power from additional replicas" against "reduced communication and more balanced load."

Three Hypotheses

- 1. Static power: Experts must reside on some GPU. The more dispersed the placement and the more replicas, the more GPUs are powered on and the higher the static power. Extra replicas bring more balanced load and reduced communication contention; the cost is higher static power consumption - a trade-off unique to MoE. It is necessary to first measure what fraction of total energy is attributable to static power to assess whether this term is worth optimizing.
- 2. Communication energy: MoE computation is sparse but all-to-all communication is dense. Communication energy is a dominant term unique to MoE, entirely absent in dense models, and constitutes the most important dynamic component in the energy model.
- 3. Latency-optimal $\neq$ energy-optimal: The configuration that minimizes latency and the configuration that minimizes energy are generally not the same. Demonstrate this empirically - a latency-energy scatter plot (latency on the x-axis, J/token on the y-axis, sweeping over multiple placement / replica / batch configurations).

Research Questions

1. RQ1: Construct an MoE inference energy model (static / dynamic compute / communication components) and empirically measure the gap between latency-optimal and energy-optimal configurations.
2. RQ2: Energy-aware expert replication & placement - under SLO constraints, determine where each expert is placed and how many hot replicas to maintain, with the objective of minimizing J/token.

Modeling

System Assumptions

- Topology: 2-tier spine-leaf datacenter network, with intra-node NVLink, intra-rack leaf+IB, and cross-rack spine.
- MoE Deployment: L layers $\times$ E experts, distributed across G GPUs via expert parallelism, with each expert permitted multiple replicas.
- Decoding: Batch size B ; each token activates k experts per layer (top- k routing).

Decision Variables

$x_{l,e,g}$ in $\{0,1\}$: whether expert e of layer l is placed on GPU g . The replica count $r_{l,e} = \sum_g x_{l,e,g}$ is derived from x ; DVFS frequency levels are deferred to future work.

Placement / Replication Tradeoff

The concrete decision surface is:

$$x_{l,e,g} \text{ in } 0,1, r_{l,e} = \sum_g x_{l,e,g}$$

Placement determines the topology tier of MoE traffic: intra-node NVLink, intra-rack leaf/IB, or cross-rack spine. Co-locating frequently co-activated experts can reduce communication energy and TBT, but may create hot GPUs and HBM pressure. Replicating hot experts reduces queueing and cross-node traffic, at the cost of extra static GPU power and HBM footprint.

The expected evidence is a Pareto frontier over latency / TBT and J/token: sweep placement, replica count, batch size, and topology tier; then report latency vs. J/token, static / compute / communication energy breakdown, and how many replicas are actually activated by decode traces.

Energy Model (per decode step)

$$E_{\text{step}} = E_{\text{static}} + E_{\text{compute}} + E_{\text{comm}}$$

- Static: $\sum_g [P^{\text{idle}}_g + \rho_g (P^{\text{TDP}}_g - P^{\text{idle}}_g)] \cdot T$, where P^{idle}/TDP are obtained from datasheets (H100 approx. 70 W / 700 W).
- Dynamic compute: $\alpha^{\text{load}}_l \cdot 1[\text{replica activated}] + \beta_l \cdot \text{token count per layer}$; α is analytically computable (weight size / HBM bandwidth x average power), β is derived from LLMCarbon's J/FLOP coefficient; activation triggers are linearized using 0-1 auxiliary variables to avoid charging weight-load energy to excess replicas that receive no tokens.
- Communication: $\sum_l \sum_{g'} D^{\text{I}}_{l \rightarrow g'}(x) \cdot c^{\text{comm}}_{g'}$. D^{I} is decoupled using expert-pair co-activation frequencies from traces (McCormick linearization); c^{comm} is expanded across three pJ/bit tiers: NVLink / leaf / spine (NVLink approx. 1.3, IB approx. 10-20, QM9700 approx. 1.5), with cross-rack roughly 8x higher than intra-node, providing a clear topological gradient signal for placement decisions.

Optimization Problem

$$\min_x (E_{\text{step}}(x))/(B) \text{ (J/token)}$$

Constraints: TBT SLO ($T(x) \leq T_{99}^{\text{SLO}}$, where T^{comm_g} reuses the receiver-port queuing upper bound from Plan A), HBM capacity, replica lower bound $\sum_g x_{l,e,g} \geq 1$. The $\rho_g \cdot T$ bilinear term is conservatively linearized using the $T = \text{SLO}$ upper bound. Solution: small-scale MILP (Gurobi), large-scale Lagrangian relaxation / greedy.

Coefficient Sources

- GPU power: NVIDIA H100 SXM5 datasheet for P^{TDP} approx. 700 W; idle power approximated from public profiling traces.
- Compute energy: LLMCarbon (Faiz et al., ICLR 2024) for the J/FLOP anchor; HBM bandwidth from the H100 datasheet for the α^{load} memory-bound term.
- Link / switch energy: NVLink and InfiniBand (HDR / NDR) per-bit energy from prior interconnect literature; switch energy reverse-engineered from NVIDIA QM9700 datasheet (total power / aggregate bandwidth).
- Linearization: McCormick (1976) envelope for bilinear $x * x$ terms.
- All coefficients are used as initial anchors; on-cluster wattmeter calibration is required before quantitative J/token claims.